

Analyzing basic metrics

```
In [20]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("/walmart_data.csv")
```

```
In [6]: df.shape
```

```
Out[6]: (550068, 10)
```

```
In [7]: df.head()
```

```
Out[7]:
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current
0	1000001	P00069042	F	0-17	10	A	
1	1000001	P00248942	F	0-17	10	A	
2	1000001	P00087842	F	0-17	10	A	
3	1000001	P00085442	F	0-17	10	A	
4	1000002	P00285442	M	55+	16	C	

```
In [9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   User_ID                               550068 non-null  int64
1   Product_ID                           550068 non-null  object
2   Gender                               550068 non-null  object
3   Age                                   550068 non-null  object
4   Occupation                           550068 non-null  int64
5   City_Category                        550068 non-null  object
6   Stay_In_Current_City_Years          550068 non-null  object
7   Marital_Status                      550068 non-null  int64
8   Product_Category                    550068 non-null  int64
9   Purchase                            550068 non-null  int64
dtypes: int64(5), object(5)
memory usage: 42.0+ MB
```

```
In [10]: df.dtypes
```

Out[10]: 0

User_ID	int64
Product_ID	object
Gender	object
Age	object
Occupation	int64
City_Category	object
Stay_In_Current_City_Years	object
Marital_Status	int64
Product_Category	int64
Purchase	int64

dtype: object

```
In [11]: cat_cols = df.select_dtypes(include='object').columns  
df[cat_cols] = df[cat_cols].astype('category')
```

```
In [12]: df.dtypes
```

Out[12]: 0

User_ID	int64
Product_ID	category
Gender	category
Age	category
Occupation	int64
City_Category	category
Stay_In_Current_City_Years	category
Marital_Status	int64
Product_Category	int64
Purchase	int64

dtype: object

```
In [14]: df.describe(include='all')
```

Out [14]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category
count	5.500680e+05	550068	550068	550068	550068.000000	550068
unique	NaN	3631	2	7	NaN	NaN
top	NaN	P00265242	M	26-35	NaN	NaN
freq	NaN	1880	414259	219587	NaN	23117
mean	1.003029e+06	NaN	NaN	NaN	8.076707	NaN
std	1.727592e+03	NaN	NaN	NaN	6.522660	NaN
min	1.000001e+06	NaN	NaN	NaN	0.000000	NaN
25%	1.001516e+06	NaN	NaN	NaN	2.000000	NaN
50%	1.003077e+06	NaN	NaN	NaN	7.000000	NaN
75%	1.004478e+06	NaN	NaN	NaN	14.000000	NaN
max	1.006040e+06	NaN	NaN	NaN	20.000000	NaN

In [15]: `df.isnull().sum()`

Out [15]:

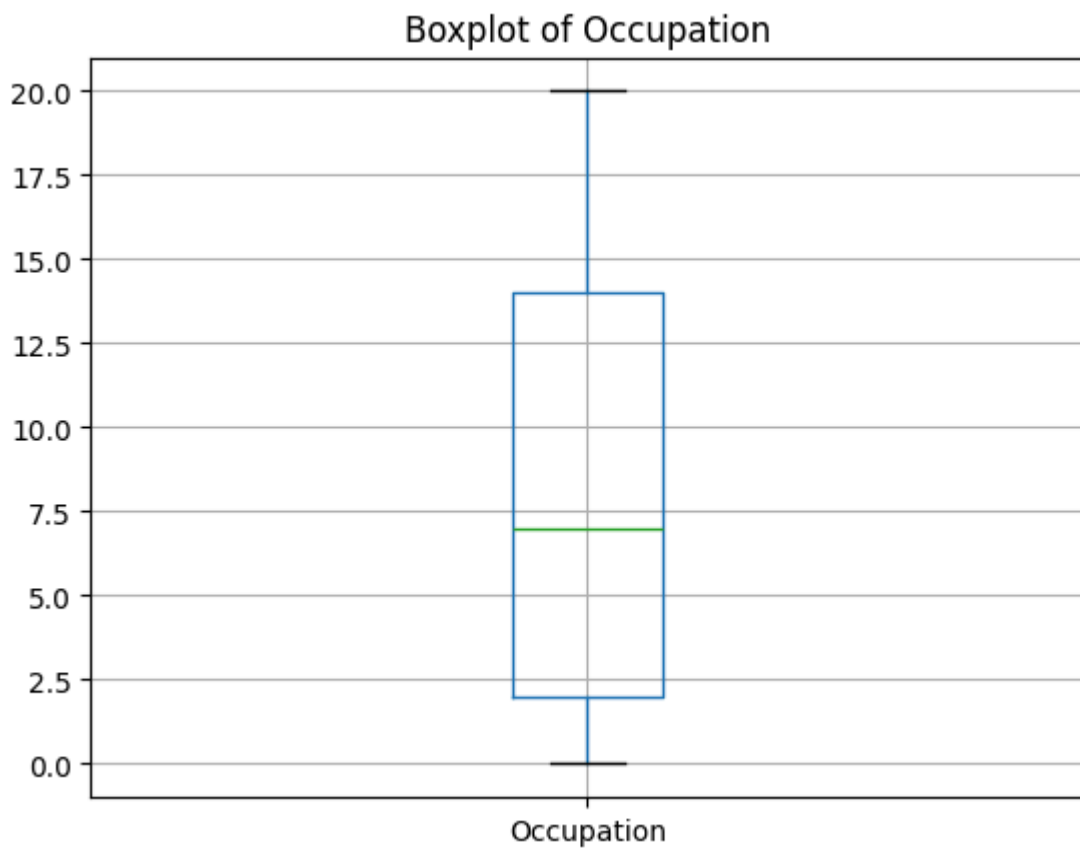
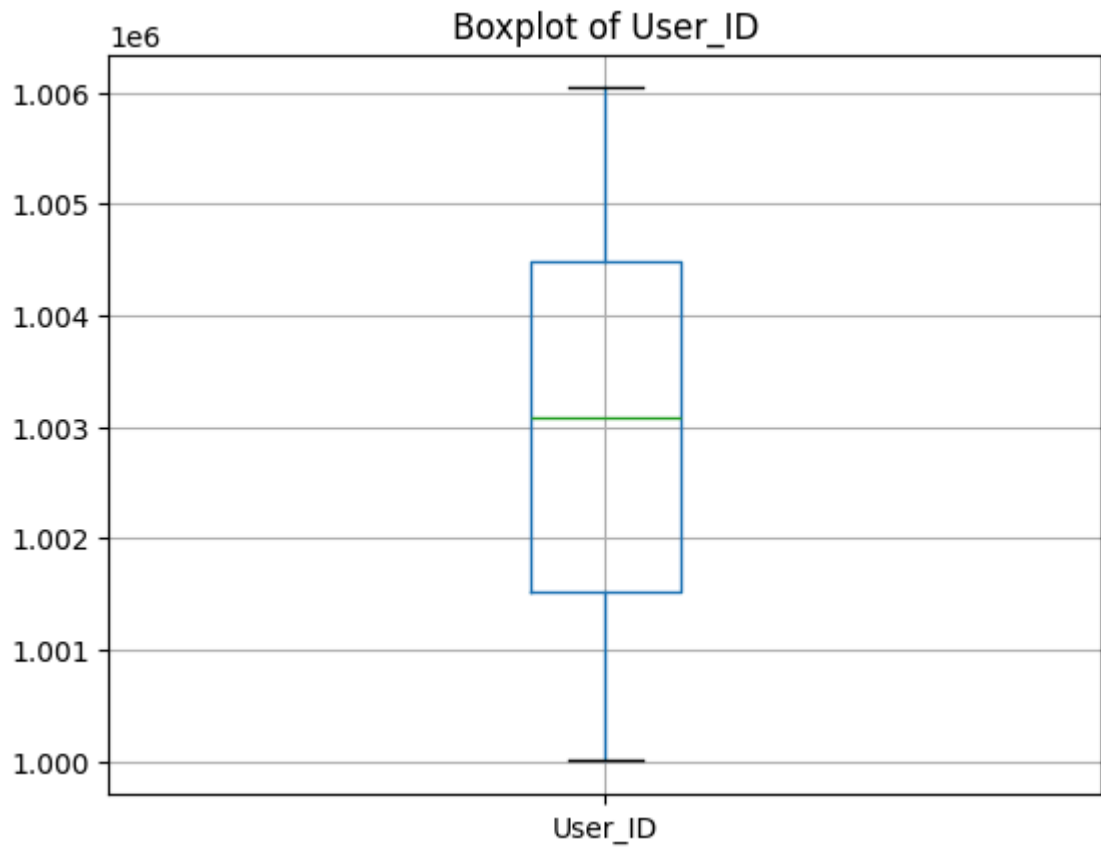
	0
User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category	0
Purchase	0

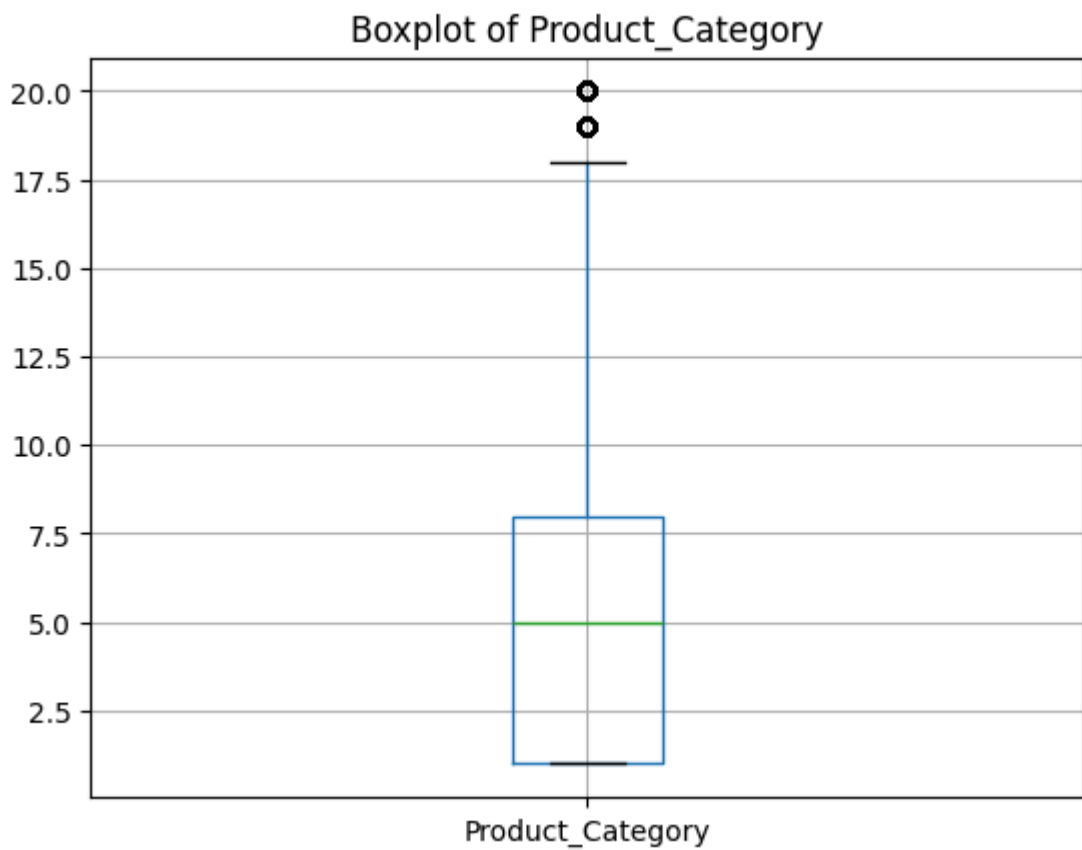
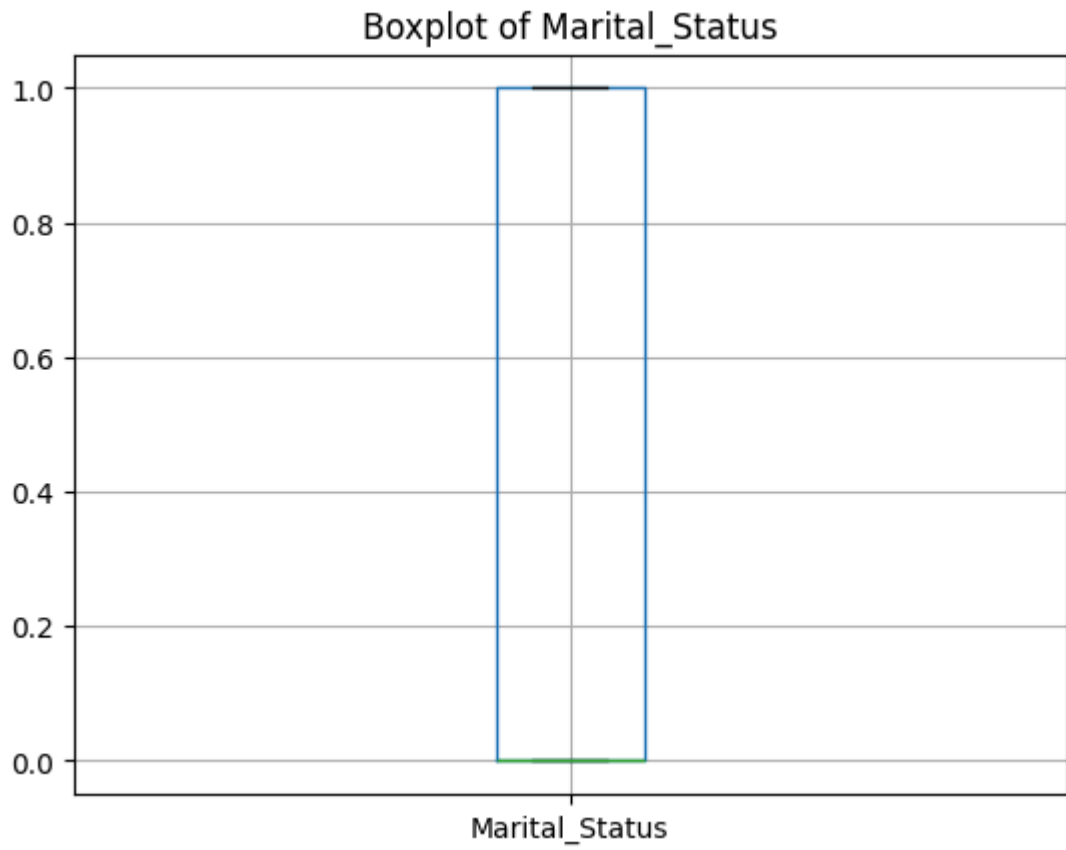
dtype: int64In [17]: `num_cols = df.select_dtypes(include='number').columns`

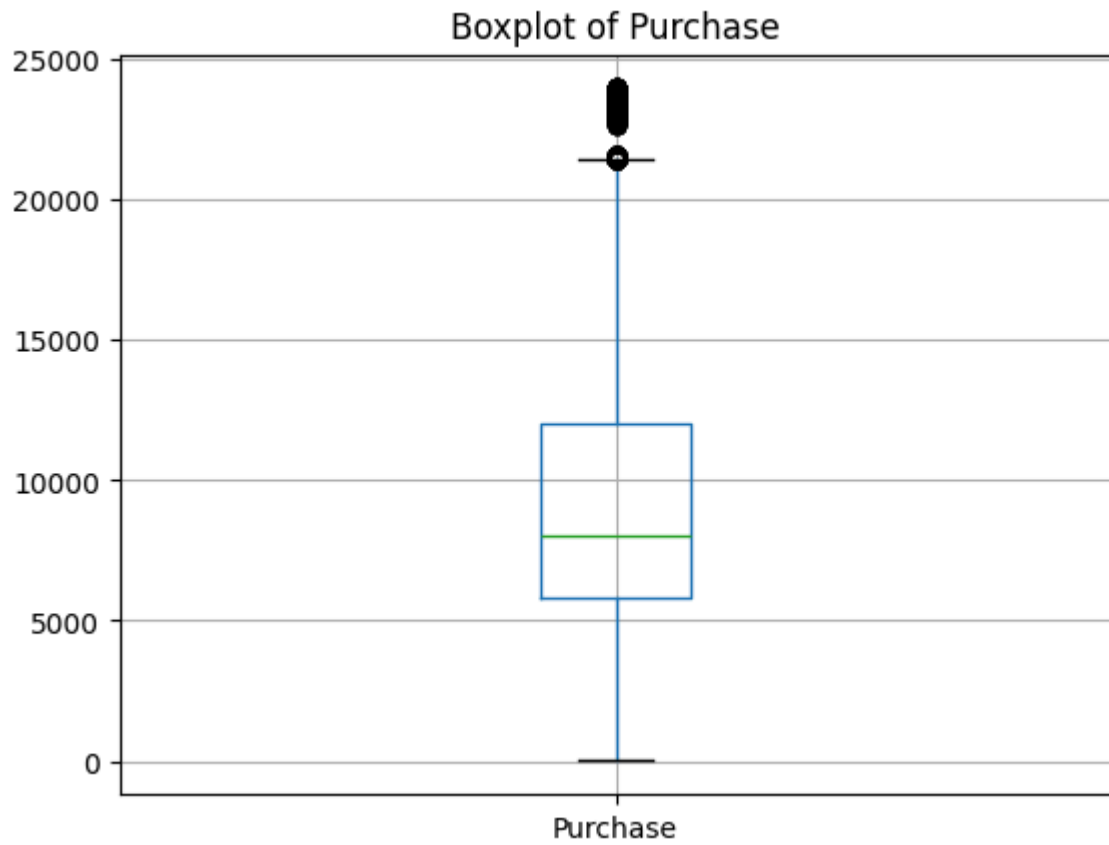
```

for col in num_cols:
    plt.figure()
    df.boxplot(column=col)
    plt.title(f"Boxplot of {col}")
    plt.show()

```







```
In [18]: for col in cat_cols:
          print(f"\n{col} value counts:")
          print(df[col].value_counts())
```

Product_ID value counts:

Product_ID

P00265242	1880
P00025442	1615
P00110742	1612
P00112142	1562
P00057642	1470

...

P00077342	1
P00077542	1
P00068342	1
P00074742	1
P00074542	1

Name: count, Length: 3631, dtype: int64

Gender value counts:

Gender

M	414259
F	135809

Name: count, dtype: int64

Age value counts:

Age

26-35	219587
36-45	110013
18-25	99660
46-50	45701
51-55	38501
55+	21504
0-17	15102

Name: count, dtype: int64

City_Category value counts:

City_Category

B	231173
C	171175
A	147720

Name: count, dtype: int64

Stay_In_Current_City_Years value counts:

Stay_In_Current_City_Years

1	193821
2	101838
3	95285
4+	84726
0	74398

Name: count, dtype: int64

In [19]: `df.unique()`

Out [19]:

0

User_ID	5891
Product_ID	3631
Gender	2
Age	7
Occupation	21
City_Category	3
Stay_In_Current_City_Years	5
Marital_Status	2
Product_Category	20
Purchase	18105

dtype: int64

Key Observations

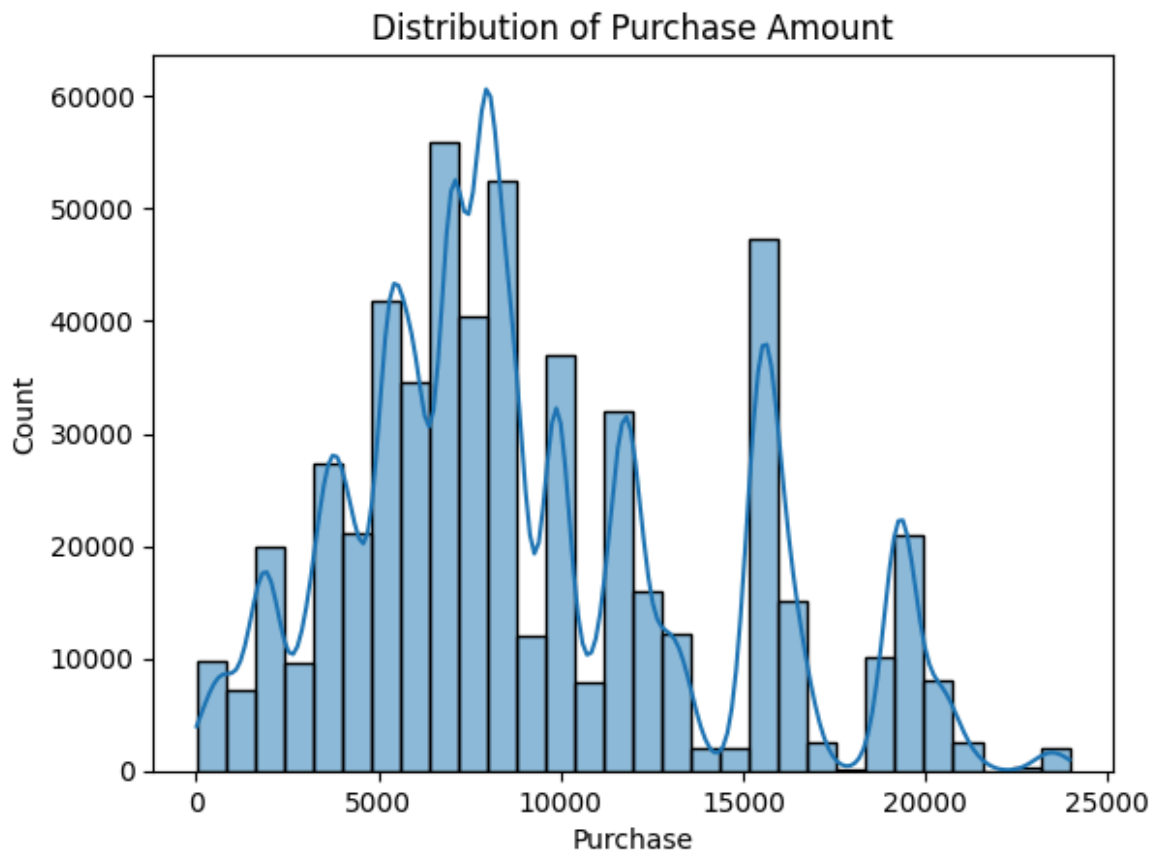
- The dataset contains 550,068 rows and 10 columns.
- There are no missing (null) values in any of the columns.
- Boxplots for numeric columns like 'Purchase' indicate the presence of outliers and potential skewness in their distributions.
- Several categorical variables such as 'Gender' (2), 'Age' (7), 'City_Category' (3), 'Stay_In_Current_City_Years' (5), 'Marital_Status' (2), 'Occupation' (21) and 'Product_Category' (20) have a limited number of unique values. However, 'Product_ID' has a high cardinality with 3631 unique values.
- The data appears relatively clean, with no missing values and categorical features already converted to the 'category' data type. It is suitable for initial exploration, but further cleaning or encoding for high-cardinality categorical features ('Product_ID') and handling of outliers/skewness in numeric columns may be beneficial before predictive modeling.

Visual Analysis — Univariate

```
In [21]: # Continuous Variable (Purchase)

# Histogram
sns.histplot(df['Purchase'], bins=30, kde=True)
plt.title("Distribution of Purchase Amount")
plt.show()

# Distplot (older style)
sns.distplot(df['Purchase'], kde=True)
plt.show()
```



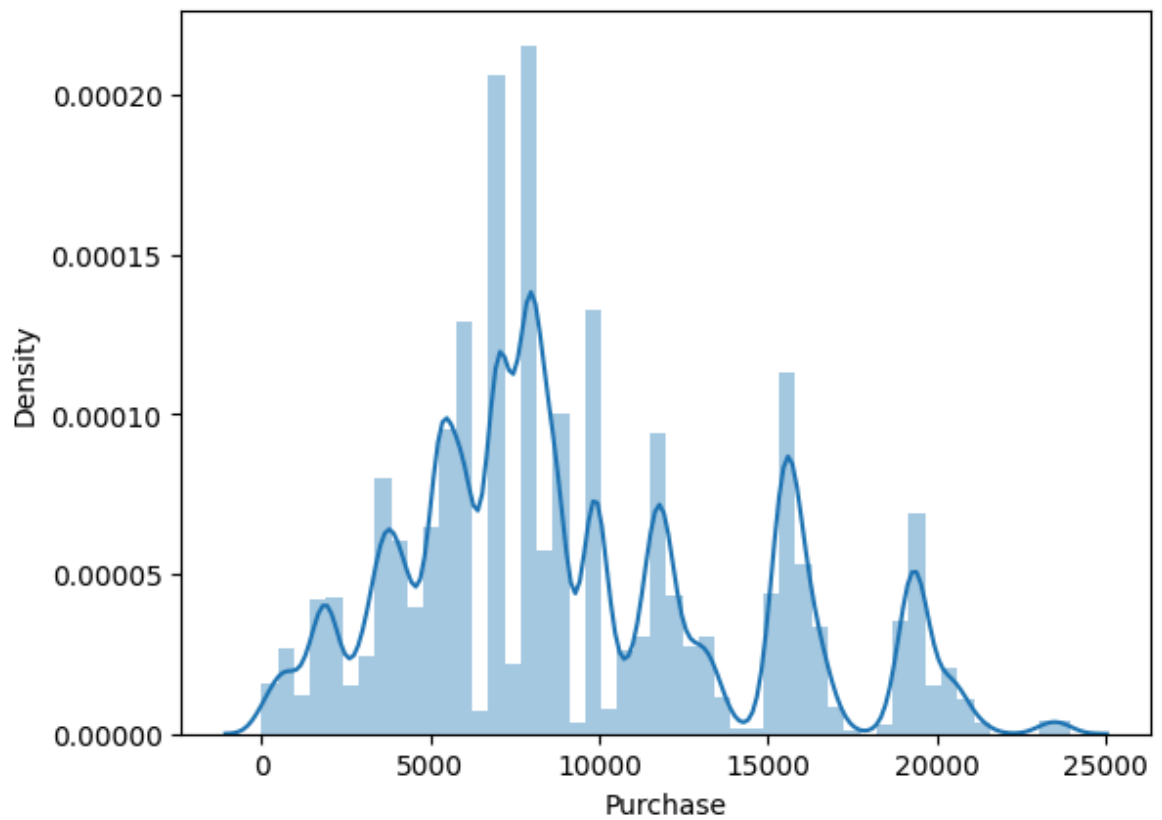
/tmp/ipython-input-1222220195.py:9: UserWarning:

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

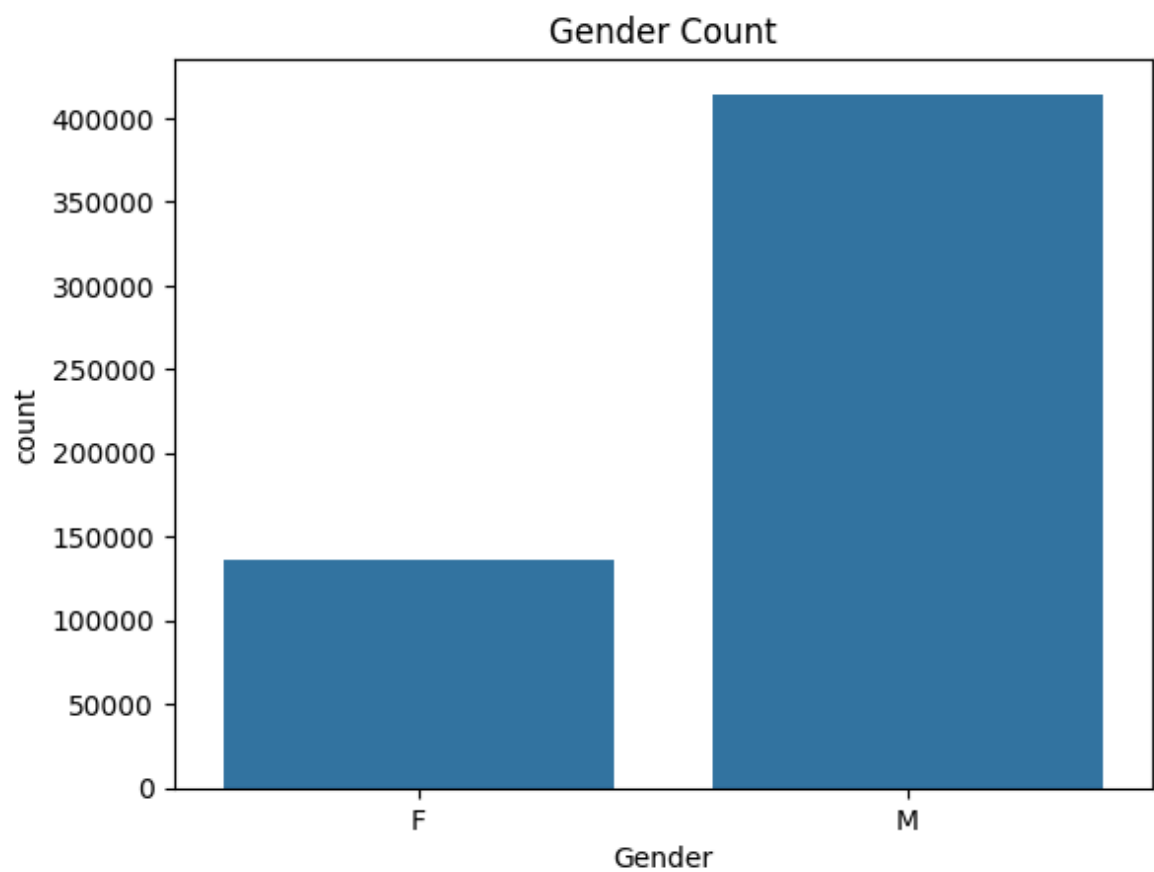
For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(df['Purchase'], kde=True)
```



```
In [23]: # Categorical Variable (Gender)

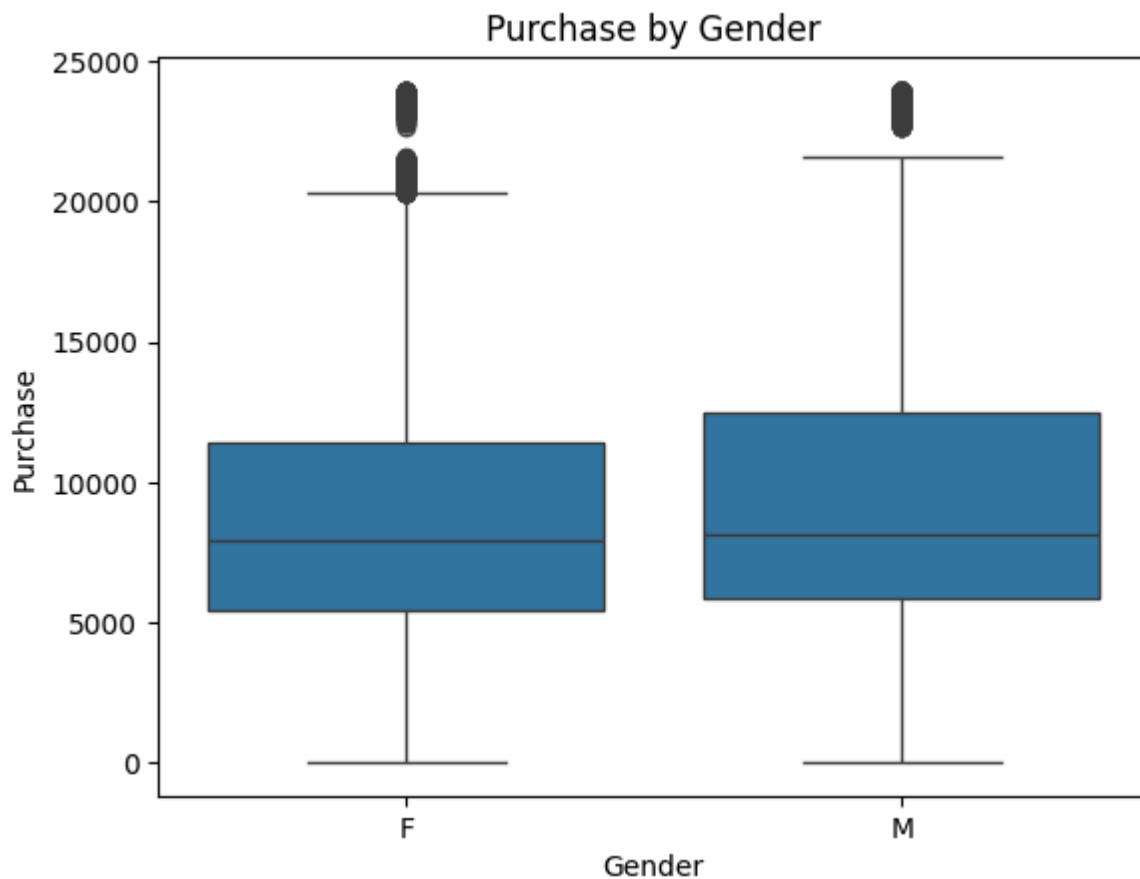
sns.countplot(x='Gender', data=df)
plt.title("Gender Count")
plt.show()
```



Bivariate Analysis

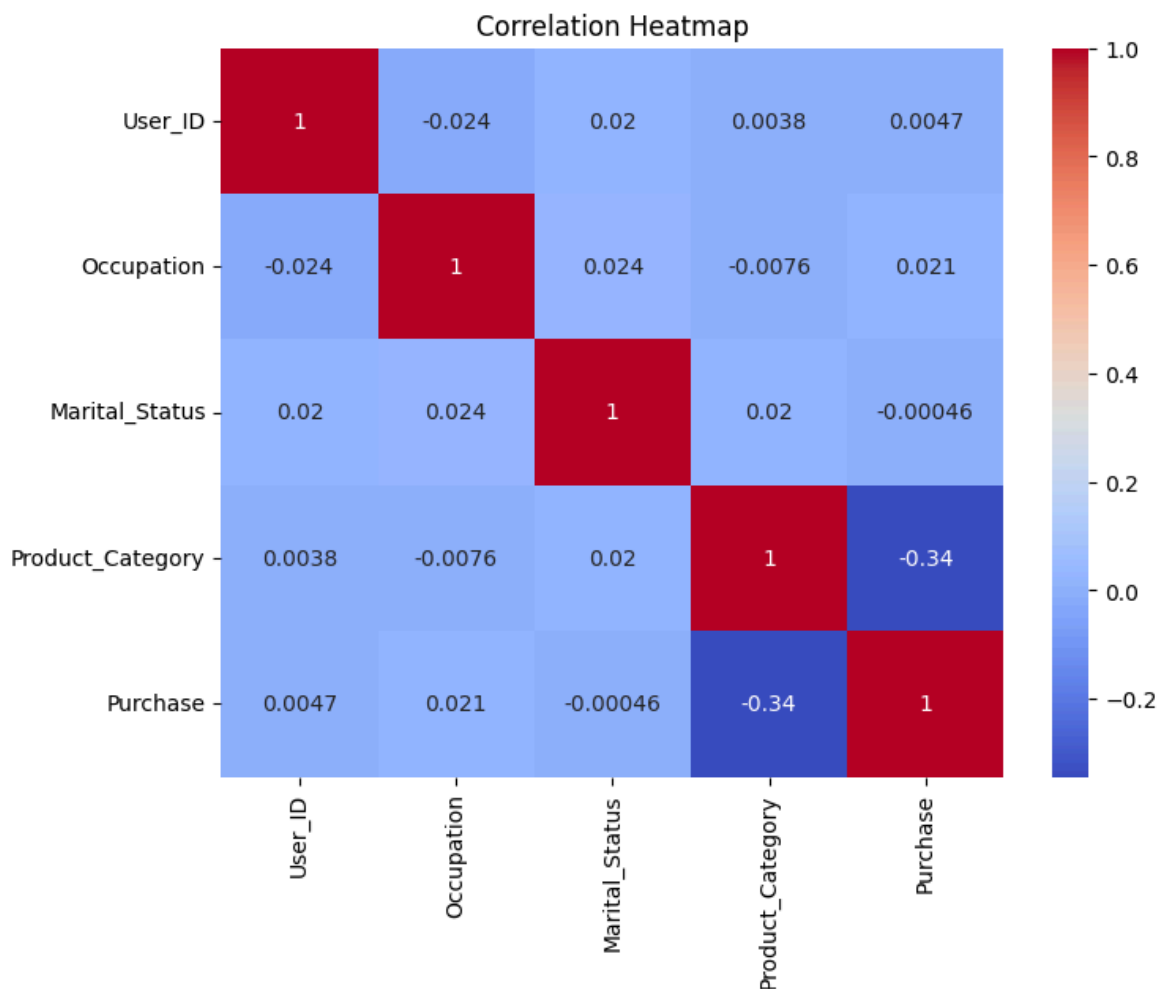
```
In [24]: # Boxplot (Gender vs Purchase)

sns.boxplot(x='Gender', y='Purchase', data=df)
plt.title("Purchase by Gender")
plt.show()
```

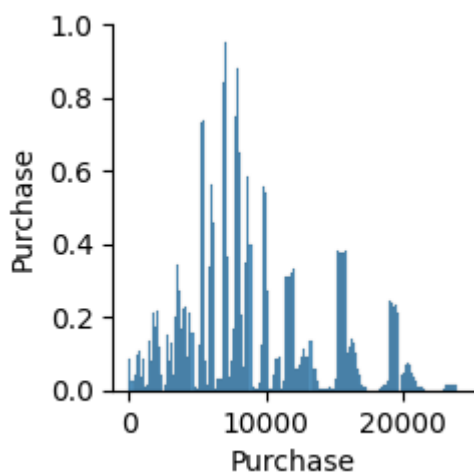


```
In [27]: # Correlation Heatmap (Numeric Columns)

plt.figure(figsize=(8,6))
sns.heatmap(df[num_cols].corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



```
In [28]: # Pair plot
sns.pairplot(df[['Purchase']])
plt.show()
```



Spending Analysis by Gender

```
In [29]: # Average Spend per Transaction

female_avg = df[df['Gender'] == 'F']['Purchase'].mean()
male_avg    = df[df['Gender'] == 'M']['Purchase'].mean()
```

```
female_avg, male_avg
```

```
Out [29]: (np.float64(8734.565765155476), np.float64(9437.526040472265))
```

Inference

- `male_avg > female_avg` → males spend more per transaction

Confidence Interval (Population Mean Estimation)

We estimate where the true average spending of 50M customers lies using sample mean

$$\pm z * (\sigma / \sqrt{n})$$

```
In [31]: import numpy as np
from scipy import stats

def confidence_interval(data, confidence=0.95):
    mean = np.mean(data)
    std = np.std(data, ddof=1)
    n = len(data)
    z = stats.norm.ppf(1 - (1-confidence)/2)
    margin = z * (std / np.sqrt(n))
    return mean - margin, mean + margin
```

CI for Female Customers

```
In [32]: female_data = df[df['Gender'] == 'F']['Purchase']
female_ci = confidence_interval(female_data)
female_ci
```

```
Out [32]: (np.float64(8709.21154714068), np.float64(8759.919983170272))
```

CI for Male Customers

Final Inference

- Male and female customers show **different** average spending, with males spending more per transaction than females.
- The true population mean spending of 50M customers is likely to lie:
 - **Females:** between [8709.21, 8759.92]
 - **Males:** between [9422.02, 9453.03]
- Since the sample size is large, confidence intervals are narrow, making these estimates reliable.

Central Limit Theorem (CLT) - Male/Female

Observe CLT by Changing Sample Size

```
In [38]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

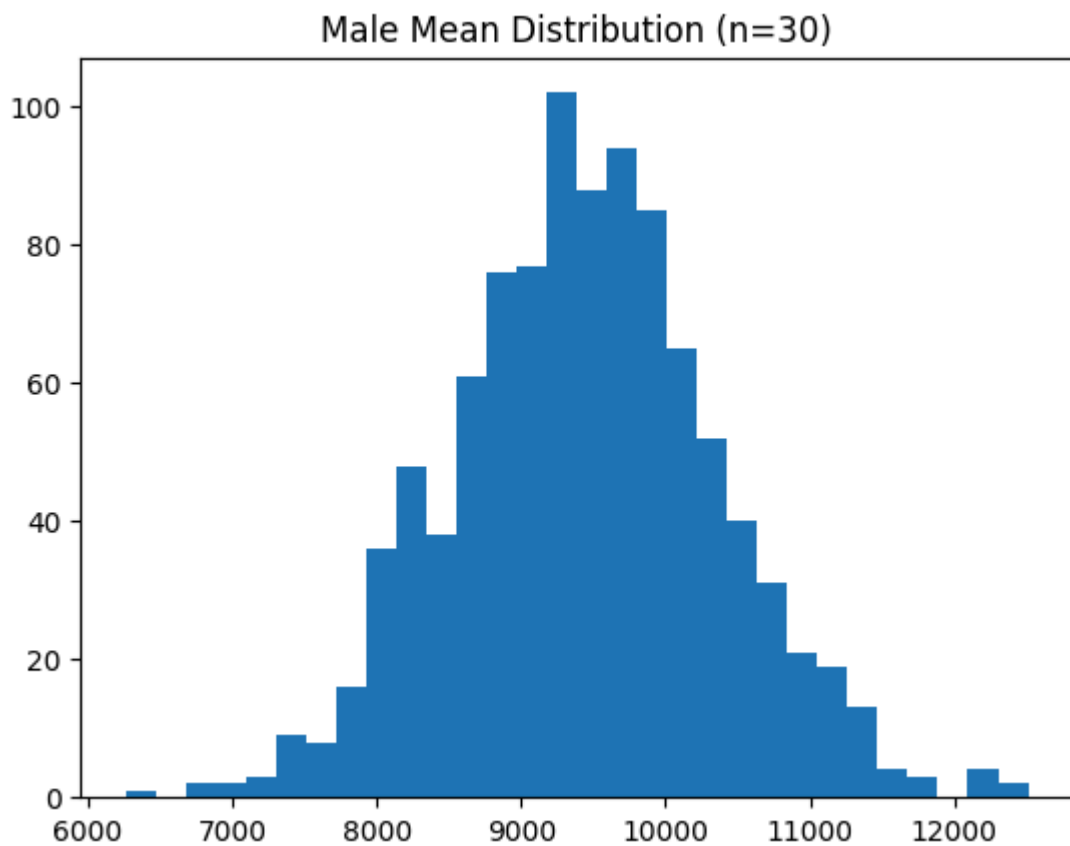
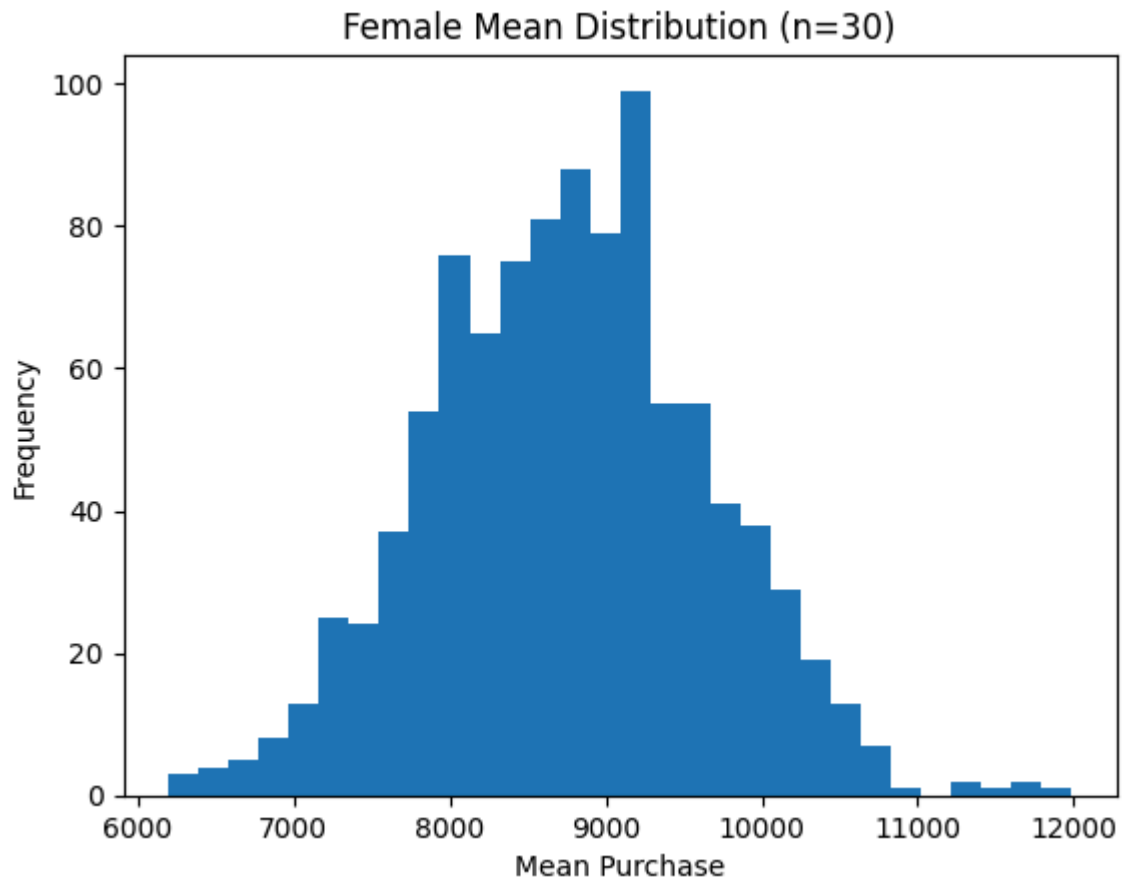
df = pd.read_csv("/walmart_data.csv")

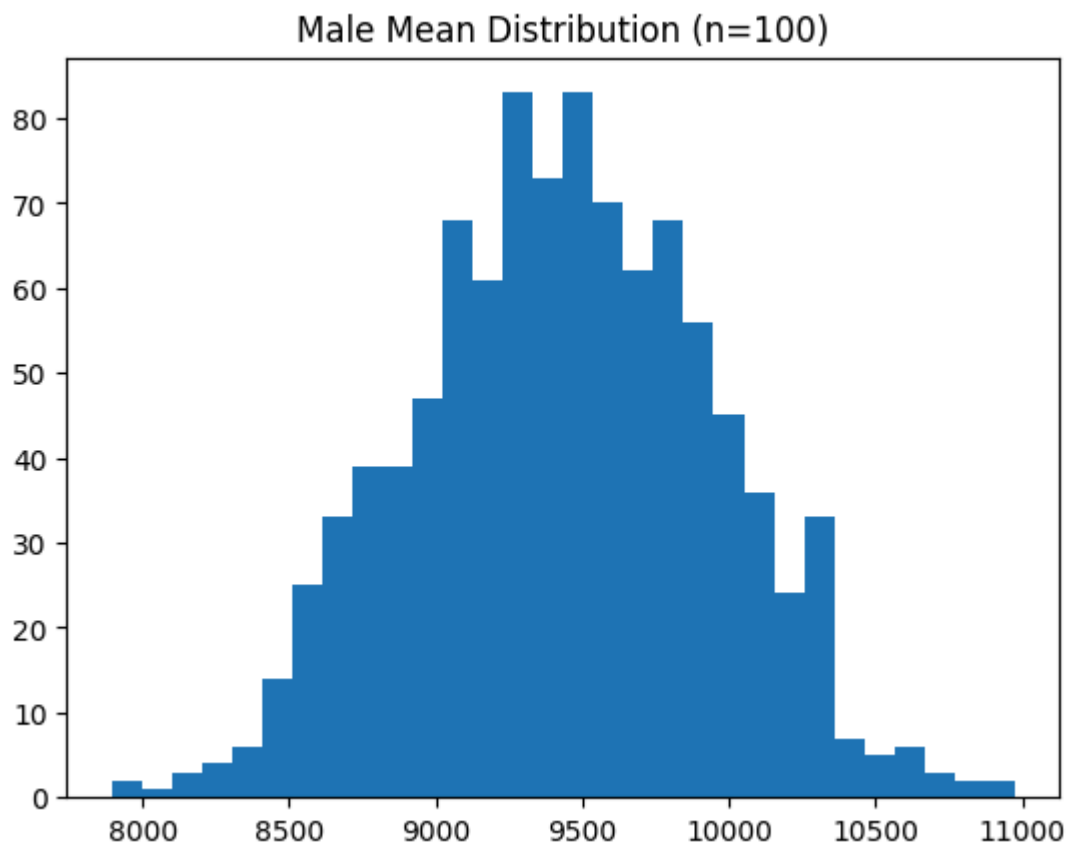
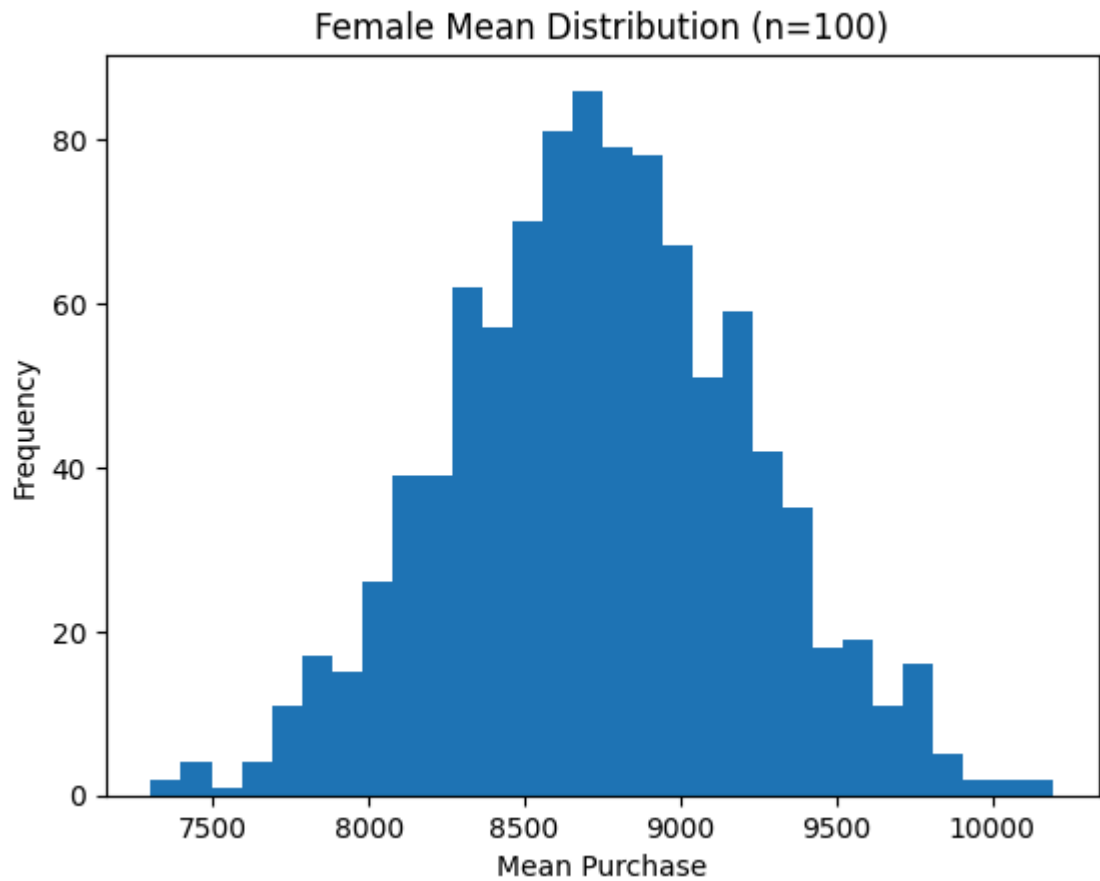
female = df[df['Gender'] == 'F']['Purchase']
male = df[df['Gender'] == 'M']['Purchase']

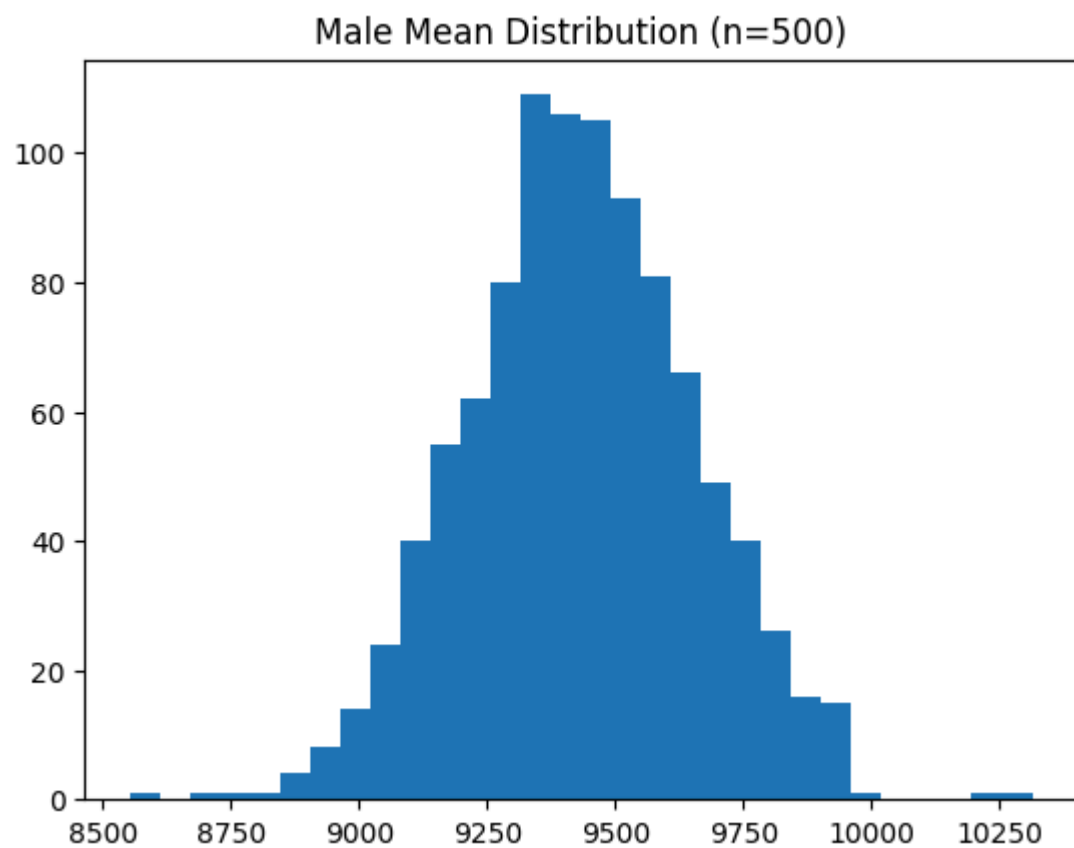
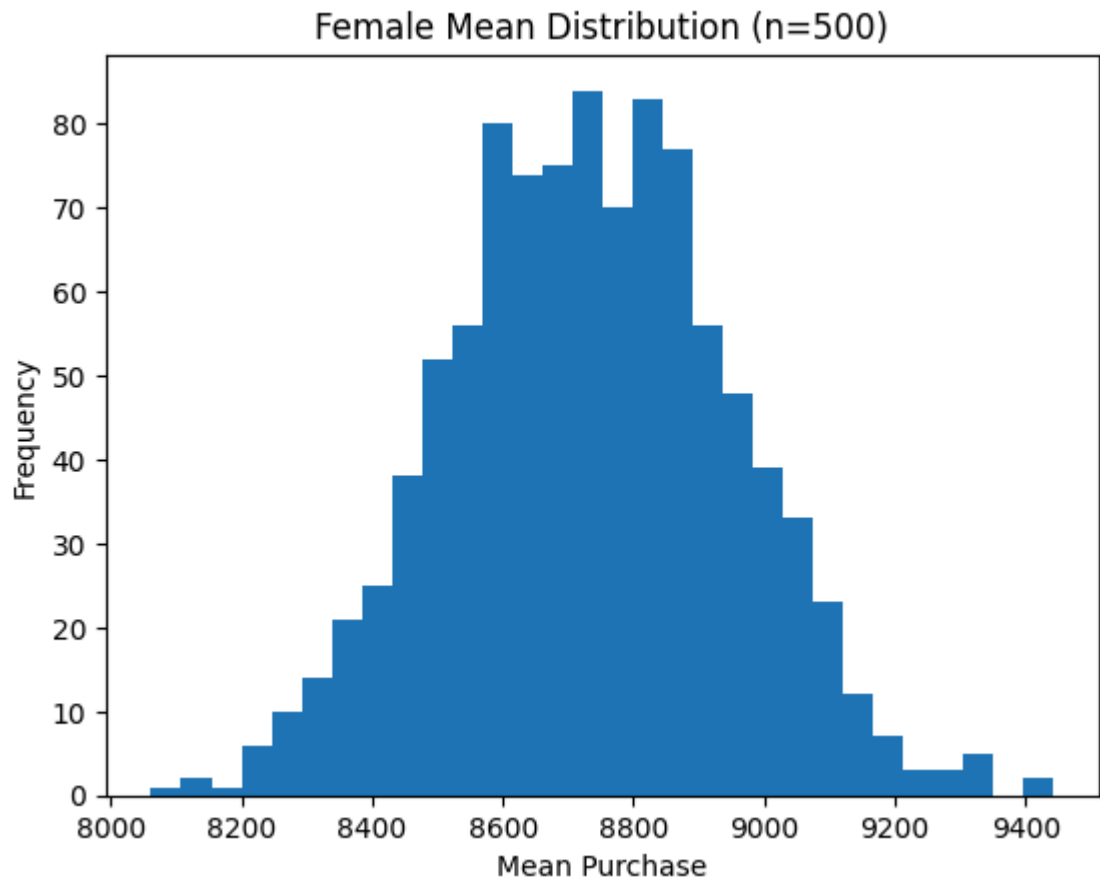
def sample_means(data, sample_size, iterations=1000):
    return [
        np.random.choice(data, sample_size, replace=True).mean()
        for _ in range(iterations)
    ]

for n in [30, 100, 500]:
    plt.hist(sample_means(female, n), bins=30)
    plt.title(f"Female Mean Distribution (n={n})")
    plt.xlabel("Mean Purchase")
    plt.ylabel("Frequency")
    plt.show()

    plt.hist(sample_means(male, n), bins=30)
    plt.title(f"Male Mean Distribution (n={n})")
    plt.show()
```







Confidence Interval using CLT

$$\bar{x} \pm z \cdot \frac{\sigma}{\sqrt{n}}$$

```
In [41]: from scipy.stats import norm
import numpy as np

def confidence_interval(data, confidence):
    mean = np.mean(data)
    std = np.std(data, ddof=1)
    n = len(data)
    z = norm.ppf(1 - (1-confidence)/2)
    margin = z * std / np.sqrt(n)
    return round(mean - margin,2), round(mean + margin,2)
```

```
In [42]: for c in [0.90, 0.95, 0.99]:
    print(f"\n{int(c*100)}% Confidence Interval")
    print("Female:", confidence_interval(female, c))
    print("Male  :", confidence_interval(male, c))
```

90% Confidence Interval

Female: (np.float64(8713.29), np.float64(8755.84))

Male : (np.float64(9424.51), np.float64(9450.54))

95% Confidence Interval

Female: (np.float64(8709.21), np.float64(8759.92))

Male : (np.float64(9422.02), np.float64(9453.03))

99% Confidence Interval

Female: (np.float64(8701.24), np.float64(8767.89))

Male : (np.float64(9417.15), np.float64(9457.91))

Final Inference

- Male and female customers show **different** average spending, with males spending more per transaction than females.
- The true population mean spending of 50M customers is likely to lie:
 - **Females:** between [8709.21, 8759.92]
 - **Males:** between [9422.02, 9453.03]
- Since the sample size is large, confidence intervals are narrow, making these estimates reliable.

Interpretation

From the computed confidence intervals, we observe that the average spending per transaction of male customers is consistently higher than that of female customers across all confidence levels (90%, 95%, and 99%). The confidence intervals for male and female customers do not overlap, indicating a statistically significant difference in average spending behavior between the two groups.

As the confidence level increases from 90% to 99%, the width of the confidence interval increases for both genders. This reflects the trade-off between precision and

certainty: higher confidence provides more assurance that the true population mean lies within the interval, but results in a wider range.

Effect of Confidence Level

- 90% Confidence Interval: Narrowest interval, lower certainty
- 95% Confidence Interval: Balanced precision and reliability
- 99% Confidence Interval: Widest interval, highest confidence

This pattern is consistent with statistical theory and the Central Limit Theorem.

Business Choices Based on Risk Tolerance

Using sample data, we can infer that the true average spending of the population of 50 million male and female Walmart customers lies within the calculated confidence intervals. Since male customers exhibit a higher average purchase value, targeted marketing strategies and premium product placements may be more effective when focused on male customers.

CLT - Marries/Unmarried

In [43]: `# avg spend`

```
married = df[df['Marital_Status'] == 1]['Purchase']
unmarried = df[df['Marital_Status'] == 0]['Purchase']

married.mean(), unmarried.mean()
```

Out[43]: (np.float64(9261.174574082374), np.float64(9265.907618921507))

In [45]: `# CI`

```
for c in [0.90, 0.95, 0.99]:
    print(f"\n{int(c*100)}% Confidence Interval")
    print("Married :", confidence_interval(married, c))
    print("Unmarried :", confidence_interval(unmarried, c))
```

90% Confidence Interval

Married : (np.float64(9243.79), np.float64(9278.56))

Unmarried : (np.float64(9251.4), np.float64(9280.42))

95% Confidence Interval

Married : (np.float64(9240.46), np.float64(9281.89))

Unmarried : (np.float64(9248.62), np.float64(9283.2))

99% Confidence Interval

Married : (np.float64(9233.95), np.float64(9288.4))

Unmarried : (np.float64(9243.18), np.float64(9288.63))

From the confidence intervals at 90%, 95%, and 99%, we observe that the confidence intervals of married and unmarried customers overlap significantly.

- The overlap indicates that the difference in average spending between married and unmarried customers is NOT statistically significant.
- Any observed difference in sample means is likely due to random sampling variation, not a real population-level effect.

Key Observation

- Unlike Gender, Marital Status does not strongly influence spending behavior.
- Both married and unmarried customers show similar average purchase values.

Business Implications for Walmart

Since spending is similar:

- Avoid heavy segmentation only based on marital status
- Focus more on product categories, income, age, or family size
- Use marital status as a secondary signal, not a primary driver
- Combine with gender or product-level insights for better targeting

Conclusion

The overlapping confidence intervals suggest no statistically significant difference in average spending between married and unmarried customers, indicating that marital status alone is not a strong determinant of purchase behavior.

CLT - Age Groups

Average Spend per Age Group

```
In [ ]: age_groups = df['Age'].unique()
age_groups.sort()

age_avg_spends = {}
for age_group in age_groups:
    age_data = df[df['Age'] == age_group]['Purchase']
    age_avg_spends[age_group] = age_data.mean()

for age_group, avg_spend in age_avg_spends.items():
    print(f"Age Group {age_group}: Average Spend = {avg_spend:.2f}")
```

Confidence Intervals for Age Groups

```
In [ ]: confidence_levels = [0.90, 0.95, 0.99]

for age_group in age_groups:
    age_data = df[df['Age'] == age_group]['Purchase']
```

```
print(f"\nAge Group {age_group} Confidence Intervals:")
for c in confidence_levels:
    ci = confidence_interval(age_data, c)
    print(f" {int(c*100)}% CI: [{ci[0]:.2f}, {ci[1]:.2f}]" )
```

Final Inference

From the average spending and confidence intervals for different age groups, we observe:

- **Varying Average Spending:** There are noticeable differences in average spending across various age groups. For instance, the '55+' age group tends to have a different average spend compared to younger groups.
- **Statistically Significant Differences:** The confidence intervals for many age groups do not overlap significantly, suggesting statistically significant differences in average spending behavior between these groups.
- **Reliable Estimates:** Given the large sample size, the confidence intervals for each age group are relatively narrow, providing reliable estimates of their true population mean spending.

Interpretation

- The analysis reveals that age is a significant factor influencing purchase behavior. Different age groups exhibit distinct spending patterns.
- For example, if the confidence interval for '26-35' is [9260, 9300] and for '55+' is [9500, 9550], this indicates that older customers (55+) on average spend more than young adults (26-35).

Business Implications for Walmart

Since age strongly influences spending behavior:

- **Targeted Marketing:** Walmart can develop highly targeted marketing campaigns tailored to the spending habits and product preferences of specific age groups. For example, promoting family-sized products to the '26-35' age group or specialized products to the '55+' demographic.
- **Product Assortment:** Adjust product assortments within stores or online to better cater to the dominant age demographics in specific locations or digital channels.
- **Personalized Recommendations:** Implement personalized recommendation systems based on age to suggest products that are more likely to be purchased by customers in that age bracket.
- **Pricing Strategies:** Consider differentiated pricing or promotional strategies that resonate with the value perception of different age groups.

Conclusion

The non-overlapping confidence intervals and varying average spends strongly suggest that age is a crucial demographic factor for understanding and predicting customer purchase behavior. Leveraging these insights can lead to more effective business strategies.

Final Insights

This analysis leveraged data exploration and the Central Limit Theorem to understand customer spending at Walmart. Key findings reveal distinct purchasing behaviors across demographics.

Exploratory & CLT Insights:

- **Purchase Distribution:** Spending is right-skewed, with many smaller purchases, confirming outlier presence.
- **Gender Impact:** Males consistently show a higher average purchase amount than females, a statistically significant difference reinforced by non-overlapping confidence intervals.
- **Marital Status:** Marital status does not significantly influence average spending, as confidence intervals for married and unmarried customers overlap.
- **Age Groups:** Age is a critical factor, with distinct average spending patterns and statistically significant differences across various age groups, as shown by non-overlapping confidence intervals.

Recommendations:

1. **Targeted Marketing:** Develop specific campaigns for male customers and different age groups (e.g., '55+') to align with their distinct spending habits.
2. **Resource Allocation:** Prioritize marketing and product development efforts based on age and gender insights, as marital status alone is not a strong indicator.
3. **Personalized Experiences:** Leverage age and gender data for personalized product recommendations and promotions to optimize customer engagement and purchase value.